

11 | 常见trait解析：标准库中的常见trait应该怎么用？

唐刚 · Rust 语言从入门到实战



你好，我是 Mike，今天我们一起学习 Rust 中的常见 trait。

前面两节课我们已经讲过 trait 在 Rust 中的重要性了，这节课就是 trait 在 Rust 标准库中的应用。Rust 标准库中包含大量的 trait 定义，甚至 Rust 自身的某些语言特性就是在这些 trait 的帮助下实现的。这些 trait 和标准库里的各种类型一起，构成了整个 Rust 生态的根基，只有了解它们才算真正了解 Rust。

注：这节课大量代码来自 [🔗 Tour of Rust's Standard Library Traits](#)，我加了必要的注解和分析。

学习完这节课的内容，你会对很多问题都豁然开朗。下面就让我们来学习标准库里一些比较常用的 trait。

标准库中的常用 trait

Default

我们来看 Default trait 的定义以及对 Default trait 的实现和使用。

[复制代码](#)

```
1 trait Default {  
2     fn default() -> Self;  
3 }
```

[复制代码](#)

```
1 struct Color(u8, u8, u8);  
2 impl Default for Color {  
3     // 默认颜色是黑色 (0, 0, 0)  
4     fn default() -> Self {  
5         Color(0, 0, 0)  
6     }  
7 }  
8  
9 fn main() {  
10     let color = Color::default();  
11     // 或  
12     let color: Color = Default::default();  
13 }
```

还有其他一些地方用到了 Default，比如 `Option<T>` 的 `unwrap_or_default()`，在类型参数上调用 `default()` 函数。

[复制代码](#)

```
1 fn paint(color: Option<Color>) {  
2     // 如果没有颜色参数传进来，就用默认颜色  
3     let color = color.unwrap_or_default();  
4     // ...  
5 }  
6  
7 // 由于default()是在trait中定义的关联函数，因此可方便的由类型参数调用  
8 fn guarantee_length<T: Default>(mut vec: Vec<T>, min_len: usize) -> Vec<T> {  
9     for _ in 0..min_len.saturating_sub(vec.len()) {  
10         vec.push(T::default()); // 这里用了 T::default() 这种形式  
11     }  
12     vec  
13 }
```

前面讲过，如果是 struct，还可以使用部分更新语法，这个时候其实是 Default 在发挥作用。

 复制代码

```
1  #[derive(Default)]
2  struct Color {
3      r: u8,
4      g: u8,
5      b: u8,
6  }
7  impl Color {
8      fn new(r: u8, g: u8, b: u8) -> Self {
9          Color {
10             r,
11             g,
12             b,
13         }
14     }
15 }
16 impl Color {
17     fn red(r: u8) -> Self {
18         Color {
19             r,
20             ..Color::default() // 注意这一句
21         }
22     }
23     fn green(g: u8) -> Self {
24         Color {
25             g,
26             ..Color::default() // 注意这一句
27         }
28     }
29     fn blue(b: u8) -> Self {
30         Color {
31             b,
32             ..Color::default() // 注意这一句
33         }
34     }
35 }
```

Rust 标准库实际给我们提供了一个标注，也就是 `#[derive()]` 里面放 Default，方便我们为结构体自动实现 Default trait。

 复制代码

```
1 #[derive(Default)]
2 struct Color {
3     r: u8,
4     g: u8,
5     b: u8
6 }
7
8 #[derive(Default)]
9 struct Color2(u8, u8, u8);
```

注意这里的细节，我们用 `#[derive()]` 在两个结构体上作了标注，这里面出现的这个 `Default` 不是 trait，它是一个同名的派生宏（我们后面会讲到）。这种派生宏标注帮助我们实现了 `Default` trait。Rustc 能正确区分 `Default` 到底是宏还是 trait，因为它们出现的位置不一样。

为什么可以自动实现 `Default` trait 呢？因为 `Color` 里面的类型是基础类型 `u8`，而 `u8` 是实现了 `Default` trait 的，默认值为 0。

Display

我们看 `Display` trait 的定义。

```
1 trait Display {
2     fn fmt(&self, f: &mut Formatter<'_>) -> Result;
3 }
```

[复制代码](#)

`Display` trait 对应于格式化符号 `"{}"`，比如 `println!("{}", s)`，用于决定一个类型如何显示，其实就是把类型转换成字符串表达。`Display` 需要我们自己手动去实现。

示例：

```
1 use std::fmt;
2 #[derive(Default)]
3 struct Point {
```

[复制代码](#)


```

4     x: i32,
5     y: i32,
6 }
7 // 为Point实现 Display
8 impl fmt::Display for Point {
9     // 实现唯一的fmt方法，这里定义用户自定义的格式
10    fn fmt(&self, f: &mut fmt::Formatter<'_,>) -> fmt::Result {
11        write!(f, "({}, {})", self.x, self.y) // write!宏向stdout写入
12    }
13 }
14
15 fn main() {
16     println!("origin: {}", Point::default());
17     // 打印出 "origin: (0, 0)"
18     // 在 format! 中用 "{}" 将类型表示/转换为 String
19     let stringified = format!("{}", Point::default());
20     assert_eq!("(0, 0)", stringified); // ✓
21 }

```

ToString

我们来看 ToString trait 定义。

📄 复制代码

```

1 trait ToString {
2     fn to_string(&self) -> String;
3 }

```

它提供了一个 to_string() 方法，方便把各种类型实例转换成字符串。但实际上不需要自己去给类型实现 ToString trait，因为标准库已经给我们做了总实现（[🔗第9讲](#)提到过），像下面这个样子。

📄 复制代码

```

1 impl<T: Display> ToString for T

```

也就是说，凡是实现了 Display 的就实现了 ToString。这两个功能本质是一样的，就是把类型转换成字符串表达。只不过 Display 侧重于展现，ToString 侧重于类型转换。下面这个示例证明这两者是等价的。

```
1 #[test] // 
2 fn display_point() {
3     let origin = Point::default();
4     assert_eq!(format!("{}", origin), "(0, 0)");
5 }
6 #[test] // 
7 fn point_to_string() {
8     let origin = Point::default();
9     assert_eq!(origin.to_string(), "(0, 0)");
10 }
11 #[test] // 
12 fn display_equals_to_string() {
13     let origin = Point::default();
14     assert_eq!(format!("{}", origin), origin.to_string());
15 }
```

所以把一个符合条件的类型实例转换成字符串有两种常用方法。

```
1 let s = format!("{}", obj);
2 // 或
3 let s = obj.to_string();
```

Debug

Debug 跟 Display 很像，也主要是用于调试打印。打印就需要指定格式，区别在于 Debug trait 是配对 "{:?}", Display 是配对 "{}" 的。它们本身都是将类型表示或转换成 String 类型。一般来说，Debug 的排版信息比 Display 要多一点，因为它是给程序员调试用的，不是给最终用户看的。Debug 还配套了一个美化版本格式 "{:#?}", 用来把类型打印得更具结构化一些，适合调试的时候查看，比如 json 结构会展开打印。

Rust 标准库提供了 Debug 宏。一般来说，我们都是以这个宏为目标类型自动生成 Debug trait，而不是由我们自己手动去实现，这一点和 Display 正好相对，std 标准库里并没有提供一个 Display 宏，来帮助我们自动实现 Display trait，需要我们手动实现它。

再提醒你一下，Rust 的类型能够自动被 derive 的条件是，它里面的每个元素都能被 derive，比如下面这个结构体里的每个字段，都是 i32 类型的，这种基础类型在标准库里已经被实现过 Debug trait 了，所以可以直接在 Point 上做 derive 为 Point 类型实现 Debug trait。这个原则适用于所有 trait，后面不再赘述。

 复制代码

```
1 #[derive(Debug)]
2 struct Point {
3     x: i32,
4     y: i32,
5 }
```

PartialEq 和 Eq

如果一个类型上实现了 PartialEq，那么它就能比较两个值是否相等。这种可比较性满足数学上的对称性和传递性，我们通过两个例子具体来看。

对称性 (symmetry) : `a == b` 导出 `b == a`。

传递性 (transitivity) : `a == b && b == c` 导出 `a == c`。

而 Eq 定义为 PartialEq 的 subtrait，在 PartialEq 的对称性和传递性的基础上，又添加了自反性，也就是对所有 `a` 都有 `a == a`。最典型的就 Rust 中的浮点数只实现了 PartialEq，没实现 Eq，因为根据 IEEE 的规范，浮点数中存在一个 NaN，它不等于自己，也就是 `NaN != NaN`。而对整数来说，PartialEq 和 Eq 都实现了。

如果一个类型，它的所有字段都实现了 PartialEq，那么使用标准库中定义的 PartialEq 派生宏，我们可以为目标类型自动实现可比较能力，用 `==` 号，或者用 `assert_eq!()` 做判断。

 复制代码

```
1 #[derive(PartialEq, Debug)]    // 注意这一句
2 struct Point {
3     x: i32,
4     y: i32,
5 }
6
```

```
7 fn example_assert(p1: Point, p2: Point) {
8     assert_eq!(p1, p2);           // 比较
9 }
10
11 fn example_compare_collections<T: PartialEq>(vec1: Vec<T>, vec2: Vec<T>) {
12     if vec1 == vec2 {             // 比较
13         // some code
14     } else {
15         // other code
16     }
17 }
```

PartialOrd 和 Ord

PartialOrd 和 PartialEq 差不多，PartialEq 只判断相等或不相等，PartialOrd 在这个基础上进一步判断是小于、小于等于、大于还是大于等于。可以看到，它就是为排序功能准备的。

PartialOrd 被定义为 PartialEq 的 subtrait。它们在类型上可以用过程宏一起 derive 实现。

[📄 复制代码](#)

```
1 #[derive(PartialEq, PartialOrd)]
2 struct Point {
3     x: i32,
4     y: i32,
5 }
6
7 #[derive(PartialEq, PartialOrd)]
8 enum Stoplight {
9     Red,
10    Yellow,
11    Green,
12 }
```

类似的，Ord 定义为 Eq + PartialOrd 的 subtrait。如果我们为一个类型实现了 Ord，那么对那个类型的所有值，我们可以做出一个严格的总排序，比如 u8，我们可以严格地从 0 排到 255，形成一个确定的从小到大的序列。

同样的，浮点数实现了 PartialOrd，但是没实现 Ord。

由于 Ord 严格的顺序性，如果一个类型实现了 Ord，那么这个类型可以被用作 BTreeMap 或 BTreeSet 的 key。

BTreeMap、BTreeSet：相对于 HashMap 和 HashSet，是两种可排序结构。

示例：

 复制代码

```
1 use std::collections::BTreeSet;
2
3 #[derive(Ord, PartialOrd, PartialEq, Eq)] // 注意这一句，4个都写上
4 struct Point {
5     x: i32,
6     y: i32,
7 }
8
9 fn example_btreeset() {
10     let mut points = BTreeSet::new();
11     points.insert(Point { x: 0, y: 0 }); // 作key值插入
12 }
13
14 // 实现了Ord trait的类型的集合，可调用 .sort() 排序方法
15 fn example_sort<T: Ord>(mut sortable: Vec<T>) -> Vec<T> {
16     sortable.sort();
17     sortable
18 }
```

运算符重载

Rust 提供了一个 Add trait，用来对加号（+）做自定义，也就是运算符重载。

你可以看一下 Add 的定义，它带一个类型参数 Rhs，这里的类型参数可以是任意名字，默认类型是 Self，一个关联类型 Output，一个方法 add()。

 复制代码

```
1 trait Add<Rhs = Self> {
2     type Output;
3     fn add(self, rhs: Rhs) -> Self::Output;
4 }
```

像下面我给出的这个示例一样去使用它就可以，非常简单。

 复制代码

```
1 struct Point {
2     x: i32,
3     y: i32,
4 }
5
6 // 为 Point 类型实现 Add trait, 这样两个Point实例就可以直接相加
7 impl Add for Point {
8     type Output = Point;
9     fn add(self, rhs: Point) -> Point {
10         Point {
11             x: self.x + rhs.x,
12             y: self.y + rhs.y,
13         }
14     }
15 }
16
17 fn main() {
18     let p1 = Point { x: 1, y: 2 };
19     let p2 = Point { x: 3, y: 4 };
20     let p3 = p1 + p2; // 这里直接用+号作用在两个Point实例上
21     assert_eq!(p3.x, p1.x + p2.x); // ✓
22     assert_eq!(p3.y, p1.y + p2.y); // ✓
23 }
```

实际上，Rust 标准库提供了一套完整的与运算符对应的 trait，你在 [🔗这里](#)可以找到可重载的运算符。你可以按类似的方式练习如何自定义各种运算符。

Clone

定义：

 复制代码

```
1 trait Clone {  
2     fn clone(&self) -> Self;  
3 }
```

这个 trait 给目标类型提供了 clone() 方法用来完整地克隆实例。使用标准库里面提供的 Clone 派生宏可以方便地为目标类型实现 Clone trait。

比如：

 复制代码

```
1 #[derive(Clone)]  
2 struct Point {  
3     x: u32,  
4     y: u32,  
5 }
```

因为每一个字段（u32 类型）都实现了 Clone，所以通过 derive，自动为 Point 类型实现了 Clone trait。实现后，Point 的实例 point 使用 point.clone() 就可以把自己克隆一份了。

通过方法的签名，可以看到方法使用的是实例的不可变引用。

 复制代码

```
1     fn clone(&self) -> Self;
```

这里面有两种情况。

第一种是已经拿到实例的所有权，clone 一份生成一个新的所有权并被局部变量所持有。

第二种是只拿到一个实例的引用，想拿到它的所有权，如果这个类型实现了 Clone trait，那么就可以 clone 一份拿到这个所有权。

clone() 是对象的深度拷贝，可能会有比较大的额外负载，但是就大多数情况来说其实还好。不要担心在 Rust 中使用 clone()，先把程序功能跑通最重要。Rust 的代码，性能一般都不会太差，毕竟起点很高。

注：浅拷贝是按值拷贝一块连续的内存，只复制一层，不会去深究这个值里面是否有到其它内存资源的引用。与之相对，深拷贝就会把这些引用对象递归全部拷贝。

在 Rust 生态的代码中，我们经常看到 clone()。为什么呢？因为它把对实例引用的持有转换成了对对象所有权的持有。一旦我们拿到了所有权，很多代码写起来就比较轻松了。

Copy

接下来，我们看 Copy trait 的定义。

```
1 trait Copy: Clone {}
```

 复制代码

定义为 Clone 的 subtrait，并且不包含任何内容，仅仅是一个标记（marker）。有趣的是，我们不能自己为自定义类型实现这个 trait。比如下面这个示例就是不行的。

```
1 impl Copy for Point {} // 这是不行的
```

 复制代码

但是 Rust 标准库提供了 Copy 过程宏，可以让我们自动为目标类型实现 Copy trait。

```
1 #[derive(Copy, Clone)]  
2 struct SomeType;
```

 复制代码

因为 Copy 是 Clone 的 subtrait。所以理所当然要把 Clone trait 也一起实现，我们在这里一次性 derive 过来。

Copy 和 Clone 的区别是，Copy 是浅拷贝只复制一层，不会去深究这个值里面是否有到其他内存资源的引用，比如一个字符串的动态数组。

 复制代码

```
1 struct Atype {
2     num: u32,
3     a_vec: Vec<u32>,
4 }
5 fn main() {
6     let a = Atype {
7         num: 100,
8         a_vec: vec![10, 20, 30],
9     };
10    let b = a; // 这里发生了移动
11 }
```

代码第 10 行的操作是将 a 的所有权移动给 b（[🔗第 2 讲](#)的内容）。

如果我们给这个结构体实现了 Clone trait 的话，我们可以调用.clone() 来产生一份新的所有权。

 复制代码

```
1 #[derive(Clone, Debug)]
2 struct Atype {
3     num: u32,
4     a_vec: Vec<u32>,    // 动态数组资源在堆内存中
5 }
6 fn main() {
7     let a = Atype {
8         num: 100,
9         a_vec: vec![10, 20, 30],
10    };
11    let mut b = a.clone(); // 克隆，也将堆内存中的Vec资源部分克隆了一份
12    b.num = 200;           // 更改b的值
13    b.a_vec[0] = 11;
14    b.a_vec[1] = 21;
15    b.a_vec[2] = 31;
16
17    println!("{a:?}"); // 对比两份值
18    println!("{b:?}");
19 }
20 // 输出
```

```
21 Atype { num: 100, a_vec: [10, 20, 30] }
22 Atype { num: 200, a_vec: [11, 21, 31] }
```

通过例子可以看到，clone() 一份新的所有权出来，b 改动的值不影响 a 的值。

而一旦你想在 Atype 上实现 Copy trait 的话，就会报错。

[复制代码](#)

```
1 error[E0204]: the trait `Copy` cannot be implemented for this type
2 --> src/main.rs:1:10
3   |
4 1 | #[derive(Copy, Clone, Debug)]
5   |           ^^^^^
6 ...
7 4 |     a_vec: Vec<u32>,    // 动态数组资源在堆内存中
8   |     ----- this field does not implement `Copy``
```

它说动态数组字段 a_vec 没有实现 Copy trait，所以你不能对 Atype 实现 Copy trait。原因也好理解，Vec 是一种所有权结构，如果你在它上面实现了 Copy，那再赋值的时候，就会出现对同一份资源的两个指向，冲突了！

一旦一个类型实现了 Copy，它就会具备一个特别重要的特性：**再赋值的时候会复制一份自身**。那么就相当于新创建一份所有权。我们来看下面这个值全在栈上的类型。

[复制代码](#)

```
1 #[derive(Clone)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 fn main() {
8     let a = Point {x: 10, y: 10};
9     let b = a; // 这里发生了所有权move, a在后续不能使用了
10 }
```

我们对 Point 实现 Clone 和 Copy。

 复制代码

```
1 #[derive(Copy, Clone)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 fn main() {
8     let a = Point {x: 10, y: 10};
9     let b = a; // 这里发生了复制, a在后续可以继续使用
10    let c = a; // 这里又复制了一份, 这下有3份了
11 }
```

仔细体会一下，现在你知道我们在第 2 讲里面讲到的复制与移动的语义区别根源在哪里了吧！

你可能会问，Point 结构体里面的字段其实全都是固定尺寸的，并且 u32 是 copy 语义的，按理说 Point 也是编译时已知固定尺寸的，为什么它默认不实现 copy 语义呢？

这其实是 Rust 设计者故意这么做的。因为 Copy trait 其实关联到赋值语法，仅仅从这个语法（let a = b;），很难一下子看出来这到底是 copy 还是 move，它是一种**隐式行为**。

而在所有权的第一设计原则框架下，Rust 默认选择了 move 语义。所以方便起见，Rust 设计者就只让最基础的那些类型，比如 u32、bool 等具有 copy 语义。而用户自定义的类型，一概默认 move 语义。如果用户想给自定义类型赋予 copy 语义内涵，那么他需要显式地在那个类型上添加 Copy 的 derive。

我们再回过头来看 Clone，一个类型实现了 Clone 后，需要显式地调用 .clone() 方法才会导致对象克隆，这就在代码里面留下了足迹。而如果一个类型实现了 Copy，那么它在使用 = 号对实例再赋值的时候就发生了复制，这里缺少了附加的足迹。这就为潜在的 Bug 以及性能的降低埋下了隐患，并且由于没有附加足迹，导致后面再回头来审查的时候非常困难。

试想，如果是`clone()`，那么我们只需要用代码搜索工具搜索代码哪些地方出现了`clone`函数就可以了。这个设计，在`Option<T>`和`Result<T, E>`的`unwrap()`系列函数上也有体现。

显式地留下足迹，是 Rust 语言设计重要的哲学之一。

至于 Copy 为什么要定义成 Clone 的 subtrait，而不是反过来，也是跟这个设计哲学相关。可以这么说，一般情况下，Rust 鼓励优先使用 Clone 而不鼓励使用 Copy，于是让开发者在 derive Copy 的时候，也必须 derive Clone，相当于多打了几个字符，多付出了一点代价。也许开发者这时会想，可能 Clone 就能满足我的要求了，能在结构体上的 derive 宏里面少打几个字符，也是一件好事儿。

还有一个原因其实是，Clone 和 Copy 在本质上其实是一样的，都是内存的按位复制，只是复制的规则有一些区别。

ToOwned

ToOwned 相当于是 Clone 更宽泛的版本。ToOwned 给类型提供了一个 `to_owned()` 方法，可以将引用转换为所有权实例。

常见的比如：

```
1 let a: &str = "123456";
2 let s: String = a.to_owned();
```

 复制代码

通过查看标准库和第三方库接口文档，你可以确定有没有实现这个 trait。

Deref

Deref trait 可以用来把一种类型转换成另一种类型，但是要在引用符号 `&`、点号操作符 `.` 或其他智能指针的触发下才会产生转换。比如标准库里最常见的 `&String` 可以自动转换到 `&str`（请回顾 [🔗第 4 讲](#)），就是因为 `String` 类型实现了 Deref trait。

```
[-] impl Deref for String source
```

```
[-] type Target = str
```

The resulting type after dereferencing.

```
[-] fn deref(&self) -> &str source
```

Dereferences the value.

还有 `&Vec<T>` 可以自动转换为 `&[T]`，也是因为 `Vec[T]` 实现了 `Deref`。

```
[-] impl<T, A> Deref for Vec<T, source
```

```
A>
```

```
where
```

```
    A: Allocator,
```

```
[-] type Target = [T]
```

The resulting type after dereferencing.

```
[-] fn deref(&self) -> &[T] source
```

Dereferences the value.

到这里，Rust 里很多魔法就开始揭开神秘面纱了。有了这些 trait 以及在各种类型上的实现，Rust 让我们可以写出顺应直觉、赏心悦目、功能强大的代码。

你还可以在标准库文档中搜索 Deref，查阅所有实现了 Deref trait 的 [implementors](#)。

这里需要提醒你一下，有人尝试 [用 Deref 机制去实现 OOP 继承](#)，但是那是徒劳和不完整的，有兴趣的话你可以看一下我给出的链接。

Drop

Drop trait 用于给类型做自定义垃圾清理（回收）。

 复制代码

```
1 trait Drop {  
2     fn drop(&mut self);  
3 }
```

实现了这个 trait 的类型的实例在走出作用域的时候，触发调用 drop() 方法，这个调用发生在这个实例被销毁之前。你可以看一下它的使用方式。

 复制代码

```
1 struct A;  
2 impl Drop for A {  
3     fn drop(&mut self){  
4         // 可以尝试在这里打印点东西看看什么时候调用  
5     }  
6 }
```

一般来说，我们不需要为自己的类型实现这个 trait，除非遇到特殊情况，比如我们要调用外部的 C 库函数，然后在 C 那边分配了资源，由 C 库里的函数负责释放，这个时候我们就要在 Rust 的包装类型（对 C 库中类型的包装）上实现 Drop，并调用那个 C 库中释放资源的函数。课程最后两讲 FFI 编程中，你会看到 Drop 的具体使用。

闭包相关 trait

标准库中有 3 个 trait 与闭包相关，分别是 FnOnce、FnMut、Fn。你可以看一下它们的定义。

 复制代码

```
1 trait FnOnce<Args> {  
2     type Output;  
3     fn call_once(self, args: Args) -> Self::Output;  
4 }  
5 trait FnMut<Args>: FnOnce<Args> {  
6     fn call_mut(&mut self, args: Args) -> Self::Output;  
7 }  
8 trait Fn<Args>: FnMut<Args> {  
9     fn call(&self, args: Args) -> Self::Output;  
10 }
```

前面我们也讲过，闭包就是一种能捕获上下文环境变量的函数。

 复制代码

```
1 let range = 0..10;  
2 let get_range_count = || range.count();
```

代码里的这个 `get_range_count` 就是闭包，`range` 是被这个闭包捕获的环境变量。

虽然它是一种函数，但是不通过 `fn` 进行定义。在 Rust 中，并不把这个闭包的类型处理成 `fn` 这种函数指针类型，而是有单独的类型定义。

那么，具体是什么类型呢？其实我们也不知道。闭包的类型是由 Rust 编译器在编译时确定的，并且在确定类型的时候要根据这个闭包捕获上下文环境变量时的行为来确定。

总的来说有三种行为（ 所有权三态再现）。

1. 获取了上下文环境变量的所有权，对应 `FnOnce`。
2. 只获取了上下文环境变量的 `&mut` 引用，对应 `FnMut`。
3. 只获取了上下文环境变量的 `&` 引用，对应 `Fn`。

根据这三种不同的行为，Rust 编译器在编译时把闭包生成为这三种不同类型中的一种。这三种不同类型的闭包，具体类型形式我们不知道，Rust 没有暴露给我们。但是 Rust 给我们暴露

了 FnOnce、FnMut、Fn 这 3 个 trait，就刚好对应于那三种类型。结合我们前面讲到的 trait object，就能在我们的代码中对那些类型进行描述了。

FnOnce 代表的闭包类型只能被调用一次，比如：

[复制代码](#)

```
1 fn main() {
2     let range = 0..10;
3     let get_range_count = || range.count();
4     assert_eq!(get_range_count(), 10); // ✓
5     get_range_count(); // ✗
6 }
```

再调用就报错了。

FnMut 代表的闭包类型能被调用多次，并且能修改上下文环境变量的值，不过有一些副作用，在某些情况下可能会导致错误或者不可预测的行为。比如：

[复制代码](#)

```
1 fn main() {
2     let nums = vec![0, 4, 2, 8, 10, 7, 15, 18, 13];
3     let mut min = i32::MIN;
4     let ascending = nums.into_iter().filter(|&n| {
5         if n <= min {
6             false
7         } else {
8             min = n; // 这里修改了环境变量min的值
9             true
10        }
11    }).collect::<Vec<_>>();
12    assert_eq!(vec![0, 4, 8, 10, 15, 18], ascending); // ✓
13 }
```

Fn 代表的这类闭包能被调用多次，但是对上下文环境变量没有副作用。比如：

[复制代码](#)

```
1 fn main() {
```



```
2 let nums = vec![0, 4, 2, 8, 10, 7, 15, 18, 13];
3 let min = 9;
4 let greater_than_9 = nums.into_iter().filter(|&n| n > min).collect::<Vec<_>>(
5 assert_eq!(vec![10, 15, 18, 13], greater_than_9); // ✓
6 }
```

另外，fn 这种函数指针，用在不需要捕获上下文环境变量的场景，比如：

[复制代码](#)

```
1 fn add_one(x: i32) -> i32 {
2     x + 1
3 }
4
5 fn main() {
6     let mut fn_ptr: fn(i32) -> i32 = add_one; // 注意这里的类型定义
7     assert_eq!(fn_ptr(1), 2); // ✓
8
9     // 如果一个闭包没有捕捉环境变量，它可以通过类型转换转成 fn 类型
10    fn_ptr = |x| x + 1; // same as add_one
11    assert_eq!(fn_ptr(1), 2); // ✓
12 }
```

From<T> 和 Into<T>

接下来，我们看 Rust 标准库中的两个关联的 trait `From<T>` 和 `Into<T>`，它们用于类型转换。`From<T>` 可以把类型 T 转为自己，而 `Into<T>` 可以把自己转为类型 T。

[复制代码](#)

```
1 trait From<T> {
2     fn from(T) -> Self;
3 }
4 trait Into<T> {
5     fn into(self) -> T;
6 }
```

可以看到它们是互逆的 trait。实际上，Rust 只允许我们实现 `From<T>`，因为实现了 `From` 后，自动就实现了 `Into`，请看标准库里的这个实现。

 复制代码

```
1 impl<T, U> Into<U> for T
2 where
3     U: From<T>,
4 {
5     fn into(self) -> U {
6         U::from(self)
7     }
8 }
```

对一个类型实现了 From 后，就可以像下面这样约束和使用。

 复制代码

```
1 fn function<T>(t: T)
2 where
3     // 下面这两种约束是等价的
4     T: From<i32>,
5     i32: Into<T>
6 {
7     // 等价
8     let example: T = T::from(0);
9     let example: T = 0.into();
10 }
```

我们来举一个具体的例子。

 复制代码

```
1 struct Point {
2     x: i32,
3     y: i32,
4 }
5 impl From<(i32, i32)> for Point {    // 实现从(i32, i32)到Point的转换
6     fn from((x, y): (i32, i32)) -> Self {
7         Point { x, y }
8     }
9 }
10 impl From<[i32; 2]> for Point {    // 实现从[i32; 2]到Point的转换
11     fn from([x, y]: [i32; 2]) -> Self {
12         Point { x, y }
13     }
14 }
```

```

15 fn example() {
16     // 使用from()转换不同类型
17     let origin = Point::from((0, 0));
18     let origin = Point::from([0, 0]);
19     // 使用into()转换不同类型
20     let origin: Point = (0, 0).into();
21     let origin: Point = [0, 0].into();
22 }

```

其实 From 是单向的。对于两个类型要互相转的话，是需要互相实现 From 的。

本身，`From<T>` 和 `Into<T>` 都隐含了所有权，`From<T>` 的 Self 是具有所有权的，`Into<T>` 的 T 也是具有所有权的。`Into<T>` 有个常用的比 `From<T>` 更自然的场景是，如果你已经拿到了一个变量，想把它变成具有所有权的值，`Into` 写起来更顺手。因为 `into()` 是方法，而 `from()` 是关联函数。

比如：

 复制代码

```

1 struct Person {
2     name: String,
3 }
4 impl Person {
5     // 这个方法只接收String参数
6     fn new1(name: String) -> Person {
7         Person { name }
8     }
9     // 这个方法可接收
10    // - String
11    // - &String
12    // - &str
13    // - Box<str>
14    // - char
15    // 这几种参数，因为它们都实现了Into<String>
16    fn new2<N: Into<String>>(name: N) -> Person {
17        Person { name: name.into() } // 调用into(), 写起来很简洁
18    }
19 }

```

TryFrom TryInto

`TryFrom<T>` 和 `TryInto<T>` 是 `From<T>` 和 `Into<T>` 的可失败版本。如果你认为转换可能会出现失败的情况，就选择这两个 trait 来实现。

[复制代码](#)

```
1 trait TryFrom<T> {
2     type Error;
3     fn try_from(value: T) -> Result<Self, Self::Error>;
4 }
5
6 trait TryInto<T> {
7     type Error;
8     fn try_into(self) -> Result<T, Self::Error>;
9 }
```

可以看到，调用 `try_from()` 和 `try_into()` 后返回的是 `Result`，你需要对 `Result` 进行处理。

FromStr

从字符串类型转换到自身。

[复制代码](#)

```
1 trait FromStr {
2     type Err;
3     fn from_str(s: &str) -> Result<Self, Self::Err>;
4 }
```

其实我们前面已经遇到过这个 trait，它就是字符串的 `parse()` 方法背后的 trait。

[复制代码](#)

```
1 use std::str::FromStr;
2
3 fn example<T: FromStr>(s: &str) {
4     // 下面4种表达等价
5     let t: Result<T, _> = FromStr::from_str(s);
6     let t = T::from_str(s);
7     let t: Result<T, _> = s.parse();
8     let t = s.parse::<T>(); // 最常用的写法
```

```
9 }
```

AsRef<T>

AsRef<T> 的定义类似下面这个样子：

[复制代码](#)

```
1 trait AsRef<T> {  
2     fn as_ref(&self) -> &T;  
3 }
```

它把自身的引用转换成目标类型的引用。和 Deref 的区别是，**deref() 是隐式调用的，而 as_ref() 需要你显式地调用 **。所以代码会更清晰，出错的机会也会更少。

AsRef<T> 可以让函数参数中传入的类型更加多样化，不管是引用类型还是具有所有权的类型，都可以传递。比如：

[复制代码](#)

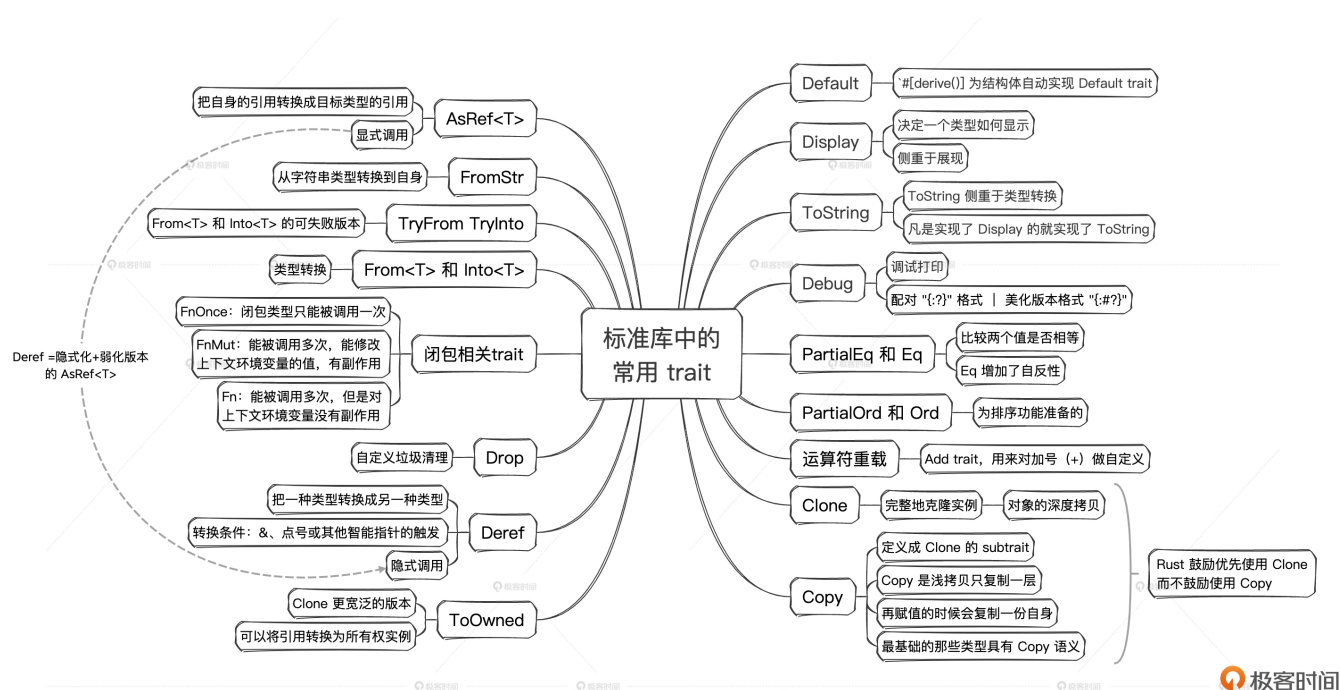
```
1 // 使用 &str 作为参数可以接收下面两种类型  
2 // - &str  
3 // - &String  
4 fn takes_str(s: &str) {  
5     // use &str  
6 }  
7 // 使用 AsRef<str> 作为参数可以接受下面三种类型  
8 // - &str  
9 // - &String  
10 // - String  
11 fn takes_asref_str<S: AsRef<str>>(s: S) {  
12     let s: &str = s.as_ref();  
13     // use &str  
14 }  
15 fn example(slice: &str, borrow: &String, owned: String) {  
16     takes_str(slice);  
17     takes_str(borrow);  
18     takes_str(owned); // ✖  
19     takes_asref_str(slice);  
20     takes_asref_str(borrow);  
21     takes_asref_str(owned); // ✔
```

在这个例子里，具有所有权的 String 字符串也可以直接传入参数中了，相对于 &str 的参数类型表达更加扩展了一步。

你可以把 Deref 看成是隐式化（或自动化）+ 弱化版本的 `AsRef<T>`。

小结

这节课我们快速过了一遍标准库里最常见的一些 trait，内容比较多。你可以先有个印象，后面遇到的时候再回过头来查阅。



这些 trait 非常重要，它们一起构成了 Rust 生态宏伟蓝图的基础。很多前面讲到的一些神奇的“魔法”都在这节课揭开了面纱。trait 这种设计真的给 Rust 带来了强大的表达力和灵活性，对它理解越深刻，越能体会 Rust 的厉害。trait 完全解构了从 C++、Java 以来编程语言的发展范式，从紧耦合转换成松散的平铺式，让新特性的添加不会对语言本身造成沉重的负担。

到这节课为止，我们的第一阶段基础篇的学习就完成了。我们用 11 讲的内容详细介绍了 Rust 语言里最重要的部分，然而还有很多 Rust 的细节没办法展开，这需要你借助我提供的链接还

有已有的资料持续学习。

基础篇相当于苦练内功，从下节课开始我们进入进阶篇，学习 Rust 语言及生态中面向实际场景的外功招式。

思考题

请举例说明 `Deref` 与 `AsRef<T>` 的区别。欢迎你把你思考后的答案分享到评论区，我们一起讨论，也欢迎你把这节课的内容分享给其他朋友，邀他一起学习 Rust，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (7)

最新 精选



Zoom 6

2024-01-04 来自江苏

`Deref` 是重载了运算符，`AsRef`是做的类型转换

作者回复: 👍



My dream

2023-12-30 来自四川

能讲一下在trait中如何使用async fn吗？我还会使用

作者回复: 可以看看这篇. <https://blog.rust-lang.org/inside-rust/2023/05/03/stabilizing-async-fn-in-trait.html>

还有刚发布的稳定版本: <https://blog.rust-lang.org/2023/12/28/Rust-1.75.0.html>

用法跟单独写 async fn 现在是一样的.



**Luke**

2023-12-22 来自上海

Deref 和 AsRef<T> 是 Rust 中用于类型转换的 trait。

Deref trait 允许你重载解引用操作符 `*`，使得一个类型可以被像指针一样解引用。这样可以方便地访问类型内部的数据。当你对一个实现了 Deref trait 的类型使用解引用操作符时，编译器会自动调用 Deref trait 中的 `deref` 方法来获取对应的值。

AsRef<T> trait 则是用于将一个类型转换为另一个类型的引用。它提供了一个 `as_ref` 方法，该方法返回一个指向目标类型的引用。这在需要将一个类型转换为另一个类型的引用时非常有用，例如将字符串转换为字节切片。

总结一下区别：

Deref trait 用于重载解引用操作符 `*`，使得一个类型可以被像指针一样解引用。

AsRef<T> trait 用于将一个类型转换为另一个类型的引用。

在使用上，如果你需要对一个类型进行解引用操作，你可以实现 Deref trait。如果你需要将一个类型转换为另一个类型的引用，你可以实现 AsRef<T> trait。

作者回复：👏

**superggn**

2023-12-15 来自北京

as_ref 没印象了，找gpt4看了看，总结了下核心就这俩：

- implicit vs explicit

- Deref trait 自动运行，``*T`` 直接触发 `deref()`，看起来像是隐式触发
- AsRef trait 需要显式调用 `as_ref()`

- 功能不同（我看到这条的时候都愣了，这谁不知道，为撒还算是 key diff）

- Deref 是解引用的
- AsRef 是创建引用的

作者回复：哈哈。gpt4有趣的。





刘丹

2023-11-13 来自广东

请问下面这一句，能否只写Ord和Eq？Ord是PartialOrd的超集，Eq是PartialEq的超集。编译器应该可以判断出，已经实现了Ord和Eq，当然也肯定实现了PartialOrd和PartialEq。

```
#[derive(Ord, PartialOrd, PartialEq, Eq)] // 注意这一句，4个都写上
```

作者回复：不能，Rust编译器就是要让你多写一点。文中有说明类似的原因。主要是怕你滥用。



杨赛军

2023-11-13 来自北京

Deref 不能传递所有权变量，Asref可以传递所有权变量

作者回复：有这个意思在里面，Deref需要通过其它操作符隐式触发，如 &, . 等，并且做的是自动 & 操作。



superggn

2023-12-15 来自北京

笔记

1. Deref trait => 给 typeA 实现了 Deref trait 之后就可以 *typeAinstance 了

2. deref coercion => 我们要对 obj_0 做一些事情的时候，如果发现 obj_0 不适配，就试一下 *obj_0, 看看它的 deref 是否适配，如果适配，就对 deref result 搞事，如果不适配，就继续找 deref, 直到当前 obj 没有 deref 为止



1